

INTERVIEW PREPARATION HANDBOOK

VelloxPrep

AI-Powered Interview Preparation Platform — Complete Technical Interview Guide

Candidate: Koushik Yenumula

Stack: Java 21 · Spring Boot 3.4 · PostgreSQL · Gemini AI · Vanilla JS

Repository: github.com/Koushikyenumula/VelloxPrep

Version: 1.0.0

Table of Contents

LEVEL 1 — PROJECT STORY

- 1.1 What is VelloxPrep?
- 1.2 Why I Built It
- 1.3 Major Features
- 1.4 Technology Stack
- 1.5 60-Second Interview Answer
- 1.6 2-Minute Interview Answer

LEVEL 2 — SYSTEM ARCHITECTURE

- 2.1 High-Level Architecture
- 2.2 Request Lifecycle
- 2.3 Backend Architecture
- 2.4 Project Structure
- 2.5 Database Architecture
- 2.6 Authentication Architecture
- 2.7 AI Integration
- 2.8 Resume Processing
- 2.9 Interview Session Lifecycle
- 2.10 Frontend Architecture
- 2.11 Deployment Architecture

LEVEL 3 — INTERVIEW DEEP DIVE

- 3.1 Java Questions
- 3.2 Spring Boot Questions
- 3.3 Spring Security Questions
- 3.4 JPA / Hibernate Questions
- 3.5 PostgreSQL Questions
- 3.6 REST API Questions
- 3.7 Gemini / AI Questions

- 3.8 Resume / ATS Questions
 - 3.9 Frontend Questions
 - 3.10 Docker / Deployment Questions
 - 3.11 Project-Specific Trap Questions (30+)
 - 3.12 "Show Me the Code" Preparation
 - 3.13 AI-Assisted Development Questions
 - 3.14 Final Interview Checklist
 - 3.15 Final Mock Interview (25 Questions)
 - Mock Interview Answer Key
-

LEVEL 1

Project Story

Confidently explain the project to any interviewer without going deep into implementation.

1.1 What is VelloxPrep?

VelloxPrep is a full-stack, AI-powered interview preparation platform built with [Java 21](#), [Spring Boot 3.4](#), [PostgreSQL](#), and [Google Gemini AI](#). It transforms passive interview studying into an active, intelligent simulation experience.

The problem it solves: Traditional interview preparation involves reading generic question-answer lists. There's no personalization, no feedback, and no way to gauge actual readiness. Candidates don't know what they're weak at until they fail a real interview.

Target users: Software engineering candidates preparing for technical interviews — backend, frontend, data science, DevOps, and related roles.

Why AI is useful: AI enables three things that static resources can't: (1) generating [tailored](#) questions based on the candidate's resume and target role, (2) [evaluating](#) free-form answers with detailed feedback and scoring, and (3) [adapting](#) difficulty based on the candidate's level.

Overall purpose: Give candidates a private, AI-powered mock interviewer they can practice with any time — covering standard interviews, resume-based interviews, MCQ quizzes, and coding challenges — while tracking progress over time.

1.2 Why I Built It

PERSONALIZE THIS

The motivation section should reflect your genuine personal story. Below is a framework — adapt it to your actual experience.

Framework for your answer:

- **Existing problem:** Interview prep resources are scattered (LeetCode for coding, random blogs for behavioral, no integrated platform). There's no personalized feedback loop.
- **Limitation of passive prep:** Reading answers doesn't build the skill of articulating them under pressure. You need practice with evaluation.
- **How VelloxPrep addresses it:** One platform that generates questions, evaluates answers with AI, tracks performance trends, and adapts to the candidate's resume and target role.
- **What makes it different:** Resume-based interview generation (questions derived from YOUR experience), real-time AI evaluation (not just right/wrong), and a full analytics dashboard.

1.3 Major Features

User Authentication (JWT + OAuth)

What it does: Users register with email/password or sign in via Google/GitHub OAuth. Each session is authenticated with a JWT token.

Why it exists: Every user needs a private account to store their resumes, sessions, and progress securely.

How it works: On login, Spring Security validates credentials against BCrypt-hashed passwords in PostgreSQL. `JwtService` generates a signed JWT with the user's email and role. The frontend stores this token in `localStorage` and sends it as `Authorization: Bearer <token>` with every API request. The `JwtAuthenticationFilter` intercepts requests, validates the token, and populates the `SecurityContext`.

Resume Upload & Management

What it does: Upload PDF resumes, view upload history, delete old resumes.

Why it exists: Resumes are the source data for personalized interview generation and ATS scoring.

How it works: Multipart file upload → validation (PDF only, ≤10MB) → saved to `uploads/` directory with UUID-prefixed filename → metadata stored in PostgreSQL → `Apache PDFBox` extracts text → `AtsScoreService` generates ATS score via Gemini → `SkillExtractionService` extracts skills via Gemini.

AI Interview Generation (Standard & Resume-Based)

What it does: Generates 10 technical interview questions based on a chosen skill/difficulty, or based on the candidate's uploaded resume.

Why it exists: Unique, targeted questions that match the candidate's actual experience and desired difficulty level.

How it works: The backend constructs a structured prompt with a system instruction telling Gemini to respond in JSON format. For resume-based interviews, extracted skills from the resume are injected into the prompt. Gemini returns a JSON array of `{question, expectedAnswer}` objects. These are parsed with Jackson's `ObjectMapper`, stored as `InterviewQuestion` entities linked to an `InterviewSession`.

AI Answer Evaluation

What it does: After submitting a free-text answer, Gemini evaluates it against the expected answer and returns a score (0–100) plus detailed feedback.

Why it exists: Transforms passive Q&A reading into an active feedback loop — the candidate knows exactly what was good, what was missing, and how to improve.

How it works: The prompt includes the question, expected answer, and user's answer. Gemini returns `{score, aiFeedback}`. The response is parsed, stored as an `AnswerEvaluation` entity, and the session's overall score is recalculated as the average of all evaluations. When all questions are answered, the session status changes to `COMPLETED` and a `Progress` record is created.

MCQ Assessment Engine

What it does: Generates multiple-choice quizzes with instant grading.

Why it exists: MCQs test breadth of knowledge quickly — useful for screening-style preparation.

How it works: Similar to interview generation — Gemini generates MCQ objects with options and correct answers. Answers are validated server-side and scored instantly.

Mock Coding Tests

What it does: Timed coding challenges where users write code, run against test cases, and submit for evaluation.

Why it exists: Simulates real coding interview environments like HackerRank/LeetCode.

How it works: Gemini generates coding problems. Users write solutions in a browser-based editor. Code is submitted, evaluated by Gemini, and results are stored in `CodingSubmission` entities.

Performance Dashboard & Progress Tracking

What it does: Displays total sessions, average scores, score trends over time, recent activity, and strongest/weakest areas.

Why it exists: Candidates need to see their improvement trajectory and focus areas.

How it works: `DashboardService` aggregates data from `InterviewSession`, `McqSession`, `CodingSession`, and `Progress` repositories. The frontend renders the data with numeric counters, charts, and session cards.

Admin Console

What it does: Platform administrators can view all users, platform statistics, promote/demote user roles, and delete users.

Why it exists: Every production platform needs administrative oversight and user management.

How it works: Endpoints under `/admin/**` are restricted to `ROLE_ADMIN` via Spring Security's `.hasRole("ADMIN")`. `AdminService` provides user listing, statistics aggregation, and role modification.

Theme System & Mobile Performance

What it does: Dark/light theme toggle with persisted preference. On mobile, the neural canvas particle engine automatically reduces from 80+ nodes to 14, caps DPR at 1, disables mouse-follower effects, and pauses rendering during scrolling.

Why it exists: Premium visual experience on desktop without sacrificing mobile performance.

1.4 Technology Stack

TECHNOLOGY	WHERE IT'S USED	WHY IT WAS CHOSEN
Java 21	Backend core language	LTS release with modern features; industry standard for enterprise backends
Spring Boot 3.4.3	Application framework	Auto-configuration, embedded server, production-ready defaults
Spring Security 6	Authentication & authorization	Industry-standard security framework with JWT integration

Spring Data JPA	Database access layer	Eliminates boilerplate SQL; auto-generates queries from method names
Hibernate 6	ORM (JPA implementation)	Maps Java objects to database tables transparently
PostgreSQL	Primary database	Reliable RDBMS; strong JSON support; free and production-proven
jjwt 0.12.6	JWT token management	Lightweight library for creating/parsing/validating JWTs
Apache PDFBox 3.0.4	Resume text extraction	Open-source Java library for reading PDF text content
Gemini AI	Question generation, answer evaluation, ATS scoring	Google's latest AI model with JSON-mode support and generous free tier
Lombok 1.18	Boilerplate reduction	@Data, @Builder, @RequiredArgsConstructor reduce verbose Java code
MapStruct 1.6	DTO ↔ Entity mapping	Compile-time code generation; faster and safer than reflection
HTML5 / CSS3 / JavaScript	Frontend	No framework overhead; full control over DOM, Canvas, and animations
Bootstrap 5	UI grid, modals, forms	Responsive grid system and pre-built UI components
Maven	Build & dependency management	Standard Java build tool; manages Spring Boot starters
Docker	Containerization	Multi-stage build for consistent deployment; platform-agnostic
Render	Cloud hosting	Free tier for backend deployment; GitHub integration

1.5 60-Second Interview Answer

INTERVIEW TIP

Speak naturally. Don't recite. Hit these points: what it is → what problem → what tech → what's unique.

"VelloxPrep is an AI-powered interview preparation platform I built using Java, Spring Boot, and Google's Gemini AI. The problem I was solving is that traditional interview prep is passive — you read questions and answers but never get feedback on whether you'd actually pass. VelloxPrep changes that by generating tailored interview questions based on your resume and target role, then evaluating your answers with AI that gives you a score and detailed feedback. It also has MCQ quizzes, coding challenges, and a performance dashboard that tracks your improvement over time. On the backend, I used Spring Security with JWT authentication, PostgreSQL for persistence, and Apache PDFBox for resume parsing. The frontend is vanilla JavaScript with a glassmorphic UI and an adaptive particle animation engine that automatically optimizes for mobile devices."

1.6 2-Minute Interview Answer

"Let me walk you through VelloxPrep. It's a full-stack AI interview prep platform.

The core idea is that most interview preparation is passive — you read generic questions from a list. There's no personalization, no feedback loop, and no way to know if you're actually improving. VelloxPrep addresses that by using Google's Gemini AI to generate role-specific questions, evaluate free-text answers in real time, and track your performance over time.

On the backend, it's a Spring Boot 3.4 application running on Java 21. I used Spring Security with stateless JWT authentication — every request carries a Bearer token that's validated by a custom `OncePerRequestFilter`. For the database, I'm using PostgreSQL with Spring Data JPA and Hibernate. The service layer follows a clean interface-implementation pattern where each service has a defined interface and a concrete implementation class.

The AI integration works through a dedicated `GeminiService` that constructs structured prompts with system instructions, calls the Gemini API via Spring's `RestClient`, parses the JSON response, and handles errors like empty responses or malformed JSON. For resume-based interviews, I use Apache PDFBox to extract text from uploaded PDFs, then feed the extracted skills into the Gemini prompt to generate targeted questions based on the candidate's actual experience.

The platform has several modules: standard interviews, resume-based interviews, MCQ assessments, and coding challenges. Each creates a session, generates questions through Gemini, evaluates answers, and stores the results. There's a dashboard that aggregates performance data across all session types.

On the frontend, it's vanilla JavaScript with Bootstrap 5 for layout and a custom visual engine that renders an animated neural canvas. I built a mobile performance optimization layer that detects touch devices and automatically reduces particle count from 80 to 14, caps DPR at 1, and pauses rendering during scroll events to maintain 60fps.

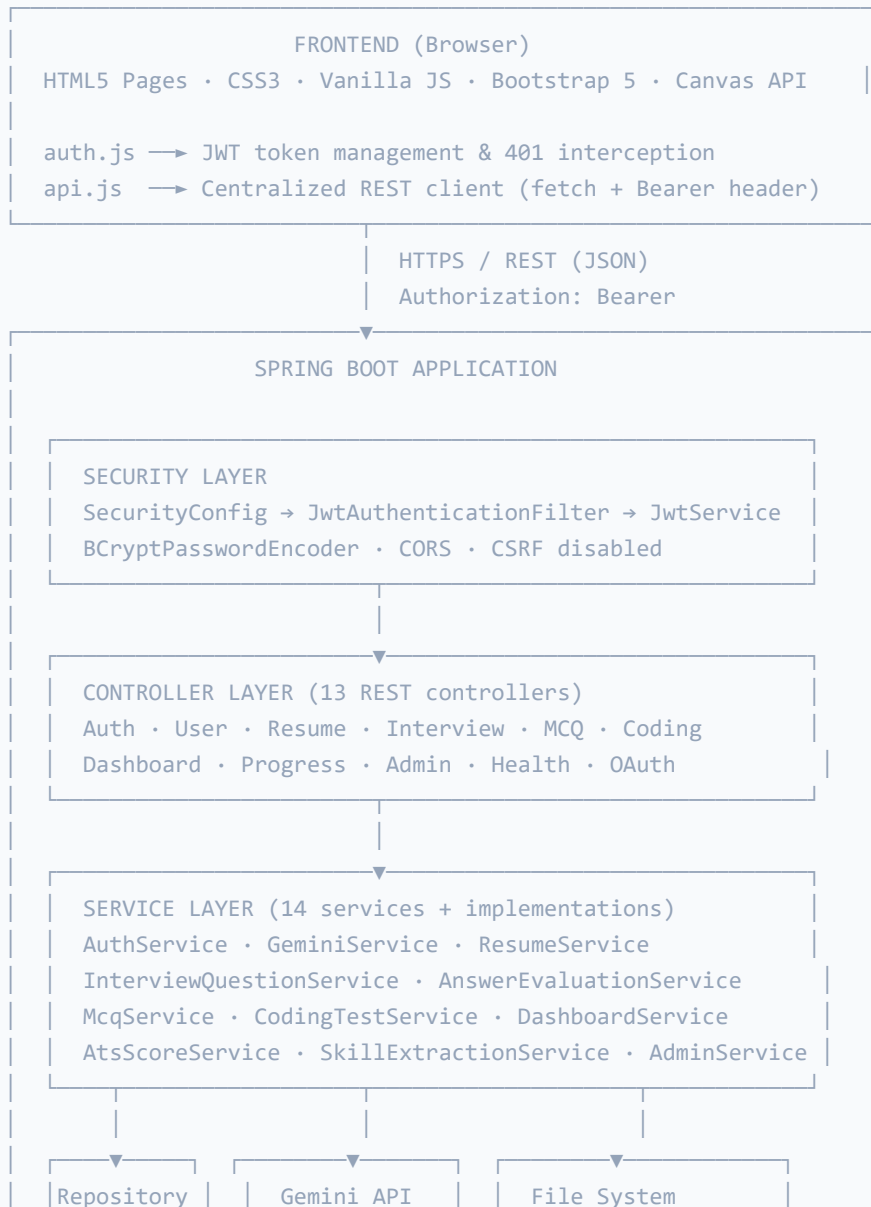
For deployment, the backend is containerized with Docker using a multi-stage build — Maven builds the JAR, then a slim JRE image runs it — and hosted on Render. The whole thing is backed by role-based access control with an admin console for user management."

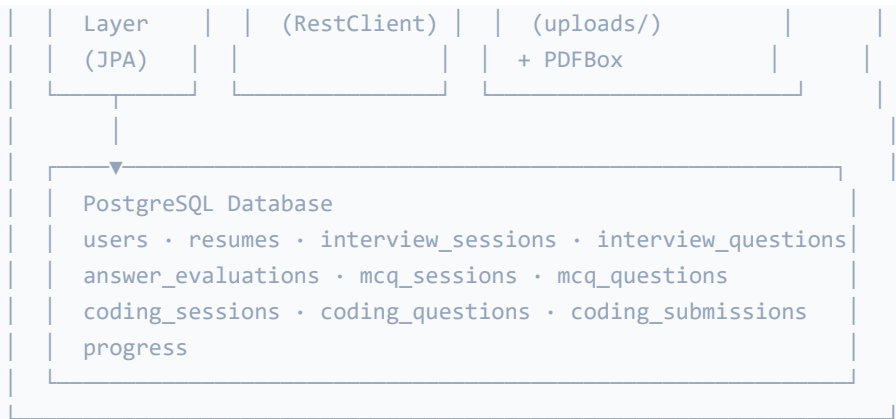
LEVEL 2

System Architecture

Understand how the system works — be able to draw the architecture and trace any request.

2.1 High-Level Architecture





2.2 Request Lifecycle

Let's trace what happens when an authenticated user generates interview questions:

```
Browser: POST /api/interviews/generate
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Body: { "skill": "Spring Boot", "difficulty": "INTERMEDIATE" }
```

1. `JwtAuthenticationFilter.doFilterInternal()`
 - Extracts token from "Authorization: Bearer ..." header
 - Calls `jwtService.isTokenValid(token)` → parses with HMAC-SHA key
 - Calls `jwtService.extractEmail(token)` → gets "john@example.com"
 - Calls `customUserDetailsService.loadUserByUsername("john@example.com")`
 - Queries `UserRepository.findByEmail()` → loads User from PostgreSQL
 - Creates `UsernamePasswordAuthenticationToken` with authorities `[ROLE_USER]`
 - Sets `SecurityContextHolder.getContext().setAuthentication(authToken)`
2. `SecurityFilterChain`
 - Checks: `/interviews/generate` → `.anyRequest().authenticated()` → ✔ PASS
 - Proceeds to controller
3. `InterviewController.generateQuestions(@RequestBody request, Principal)`
 - Extracts email from `SecurityContext`
 - Delegates to `interviewQuestionService.generateQuestions(request, email)`
4. `InterviewQuestionServiceImpl.generateQuestions()`
 - Validates difficulty enum (`BEGINNER` | `INTERMEDIATE` | `ADVANCED`)
 - Loads User from `UserRepository`
 - Builds prompt: "Generate 10 questions for Spring Boot at INTERMEDIATE..."
 - Calls `geminiService.generateContent(SYSTEM_INSTRUCTION, prompt)`

```

|   └─ GeminiServiceImpl:
|       └─ Builds JSON body with system_instruction, contents, generationConfig
|       └─ POST to googleapis.com/v1beta/models/gemini-3.1-flash-lite:generateContent
|       └─ Parses response: candidates[0].content.parts[0].text
|       └─ Returns JSON string
|   └─ Parses JSON array into InterviewQuestion entities via ObjectMapper
|   └─ Creates InterviewSession entity (status: IN_PROGRESS)
|   └─ Links questions to session via session.addQuestion()
|   └─ Saves to PostgreSQL via sessionRepository.save(session)
|   └─ Returns ApiResponse<GenerateQuestionsResponse>
|
▼
5. Response: 200 OK
{
  "status": 200,
  "message": "Questions generated successfully",
  "data": { "sessionId": 42, "questions": [...] },
  "timestamp": "2026-08-21T14:30:00"
}

```

2.3 Backend Architecture

Controller Layer

Controllers are **thin entry points** — they handle HTTP mapping, extract parameters, and delegate to services. They should **never contain business logic**.

Why separate from services? (1) Single Responsibility — controllers handle HTTP, services handle logic. (2) Testability — services can be unit-tested without HTTP concerns. (3) Reusability — the same service can be called from different controllers, scheduled tasks, or message listeners.

VelloxPrep has 13 controllers, each annotated with `@RestController` and `@RequestMapping`.

Service Layer

Each service has an **interface** (e.g., `AuthService`) and an **implementation** (e.g., `AuthServiceImpl`). This follows the **Dependency Inversion Principle** — controllers depend on abstractions, not concrete classes.

Key services and their responsibilities:

- `AuthServiceImpl` — Registration (BCrypt hashing) and login (password verification + JWT generation)
- `GeminiServiceImpl` — Builds HTTP requests to Gemini API, parses responses, handles errors

- `InterviewQuestionServiceImpl` — Prompt construction, AI question generation, JSON parsing, session creation
- `AnswerEvaluationServiceImpl` — Sends user answers to Gemini for scoring, updates session scores
- `ResumeServiceImpl` — PDF validation, file storage, metadata persistence, triggers ATS analysis

Repository Layer

Repositories extend `JpaRepository<Entity, Long>`. Spring Data JPA auto-generates SQL from method names like `findByEmail(String email)` or `findByUserEmail(String email)`. No SQL is written manually.

Entity Layer

JPA entities are Java classes annotated with `@Entity` and `@Table`. Each entity maps to a PostgreSQL table. Relationships are defined with `@OneToMany`, `@ManyToOne`, and `@JoinColumn`. Hibernate's `ddl-auto: update` auto-creates/modifies tables from entity definitions.

DTO Layer

DTOs (Data Transfer Objects) separate the API contract from the database model. Benefits: (1) Don't expose internal fields (password, audit columns). (2) API can evolve independently of the database. (3) Input DTOs carry validation annotations (`@NotBlank`, `@Email`).

Exception Layer

`GlobalExceptionHandler` is annotated with `@RestControllerAdvice`. It catches specific exceptions and returns consistent `ApiResponse` JSON. For example, `InvalidCredentialsException` → 401, `ResourceNotFoundException` → 404, `GeminiApiException` → 503, `AccessDeniedException` → 403.

2.4 Project Structure (Key Classes)

config/SecurityConfig.java

Defines the `SecurityFilterChain` bean: CORS configuration, CSRF disabled, stateless sessions, public vs authenticated endpoints, admin role restriction, and JWT filter injection before `UsernamePasswordAuthenticationFilter`.

config/GeminiConfig.java

Creates a `RestClient` bean pre-configured with the Gemini base URL

(`https://generativelanguage.googleapis.com`).

security/JwtService.java

Three core methods: `generateToken(email, role)` , `extractEmail(token)` , `isTokenValid(token)` . Uses HMAC-SHA signing with a Base64-decoded secret key.

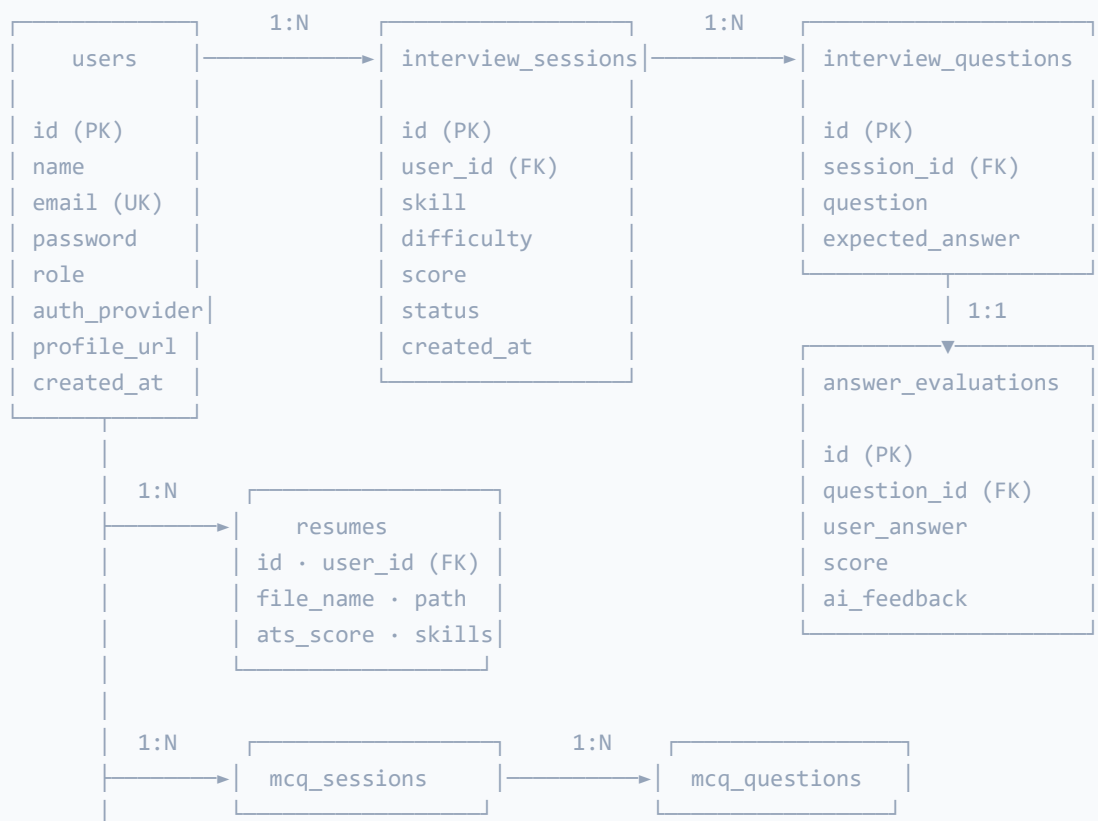
security/JwtAuthenticationFilter.java

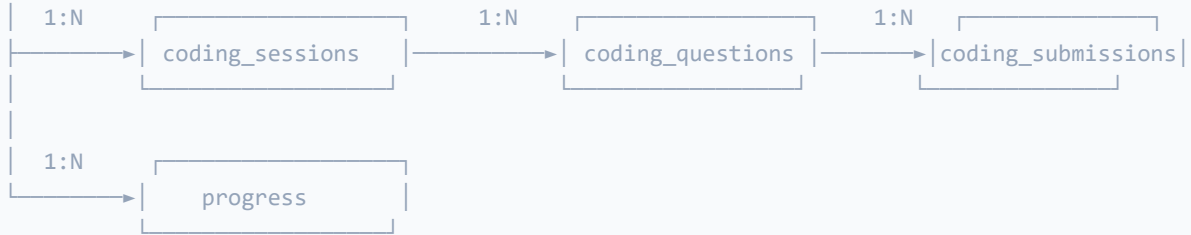
Extends `OncePerRequestFilter` . The 7-step process: extract token → validate → extract email → check SecurityContext is empty → load UserDetails → create auth token → set SecurityContext.

service/impl/GeminiServiceImpl.java

Builds JSON request bodies with Jackson's `ObjectMapper` . Calls `/v1beta/models/{model}:generateContent?key={apiKey}` . Parses `candidates[0].content.parts[0].text` from the response. Sets `responseMimeType: "application/json"` in generation config to get structured JSON back.

2.5 Database Architecture





Key relationships: User @OneToMany → InterviewSession (with CascadeType.ALL , orphanRemoval = true). InterviewSession @OneToMany → InterviewQuestion. InterviewQuestion @OneToOne → AnswerEvaluation. All relationships use FetchType.LAZY with @BatchSize(size = 20/50) for N+1 optimization.

2.6 Authentication Architecture (Step-by-Step)

- Registration:** POST /auth/register → AuthServiceImpl checks if email exists → hashes password with passwordEncoder.encode() (BCrypt) → saves User entity with Role.USER
- Login:** POST /auth/login → finds user by email → passwordEncoder.matches(rawPassword, hashedPassword) → if valid, jwtService.generateToken(email, role)
- JWT Generation:** Jwts.builder().subject(email).issuer("ai-interview-platform").expiration(now + 86400000ms).claims(Map.of("role", role)).signWith(hmacKey).compact()
- Client Storage:** Frontend stores token in localStorage.setItem('token', data.token)
- Authenticated Request:** auth.js injects Authorization: Bearer {token} header on every fetch() call
- Server Validation:** JwtAuthenticationFilter extracts token → jwtService.isTokenValid() parses and checks expiry → extracts email → loads UserDetails from DB → creates UsernamePasswordAuthenticationToken → sets SecurityContextHolder
- Authorization:** SecurityConfig checks: /admin/** → hasRole("ADMIN") , all other authenticated endpoints → user must have valid JWT

KEY CONCEPTS TO KNOW

BCrypt: Adaptive hash function with salt — same password produces different hashes each time.

JWT Structure: Header (alg) . Payload (sub, role, exp, iss) . Signature (HMAC-SHA).

Stateless Auth: Server stores NO session. Every request carries full authentication proof in the JWT.

OncePerRequestFilter: Guarantees the filter runs exactly once per HTTP request, even with forwards/includes.

2.7 AI Integration (Gemini)

```

User Input (skill, difficulty)
|
▼
InterviewQuestionServiceImpl
| Constructs prompt:
|   System: "You are a technical interview question generator.
|           Respond ONLY with a valid JSON array..."
|   User: "Generate 10 questions for Spring Boot at INTERMEDIATE..."
|
▼
GeminiServiceImpl.generateContent(systemInstruction, userPrompt)
| Builds JSON request body (ObjectMapper):
|   { system_instruction, contents, generationConfig }
| generationConfig includes:
|   temperature: 0.7, maxOutputTokens: 2048,
|   responseMimeType: "application/json"
|
▼
RestClient.post()
→ POST https://generativelanguage.googleapis.com
    /v1beta/models/gemini-3.1-flash-lite:generateContent?key={API_KEY}
|
▼
Gemini API returns:
{ "candidates": [ { "content": { "parts": [ { "text": "[{...}, ...]" } ] } } ] }
|
▼
parseResponse() extracts candidates[0].content.parts[0].text
|
▼
InterviewQuestionServiceImpl parses JSON array into List<InterviewQuestion>
|
▼
sessionRepository.save(session) → PostgreSQL
|
▼
ApiResponse<GenerateQuestionsResponse> → Client

```

Why backend, not frontend? (1) API key protection — never expose in client-side code. (2) Response validation and sanitization. (3) Server-side persistence of generated content.

Error handling: `GeminiServiceImpl` catches `RestClientException` and wraps it in `GeminiApiException`. The `parseResponse()` method checks for API-level errors, empty candidates, empty parts, and empty text — throwing specific `GeminiApiException` messages for each case.

2.8 Resume Processing

```

Client uploads PDF (multipart/form-data)
|
▼
ResumeController → ResumeServiceImpl.uploadResume(file, email)
|
├─ 1. validateFile(): checks not null, PDF content type, ≤10MB
├─ 2. UUID.randomUUID() + original filename → unique path
├─ 3. Files.createDirectories(uploadPath) → creates uploads/ if needed
├─ 4. Files.copy(inputStream, targetLocation) → saves to disk
├─ 5. resumeRepository.save(resume) → stores metadata in PostgreSQL
├─ 6. atsScoreService.analyzeResume(id) → calls Gemini for ATS score
├─ 7. skillExtractionService.extractSkills(id) → calls Gemini for skills

```

2.9 Interview Session Lifecycle

1. **Create Session**: User configures skill + difficulty → POST `/interviews/generate`
2. **Generate Questions**: Service constructs prompt → Gemini returns 10 Q&A pairs → parsed and stored as `InterviewQuestion` entities → Session status: `IN_PROGRESS`
3. **Display Questions**: Frontend fetches session via GET `/interview-sessions/{id}` → renders Q&A interface
4. **Submit Answer**: User types answer → POST `/interview/answer` with `{questionId, answer}`
5. **AI Evaluation**: Service builds evaluation prompt (question + expected + user answer) → Gemini returns `{score, aiFeedback}` → stored as `AnswerEvaluation`
6. **Calculate Results**: After all questions answered → average all scores → update session `score` → set status to `COMPLETED`
7. **Update Progress**: `Progress` record created linking user, skill, score, and timestamp
8. **Dashboard Update**: Next dashboard load aggregates latest session data

2.10 Frontend Architecture

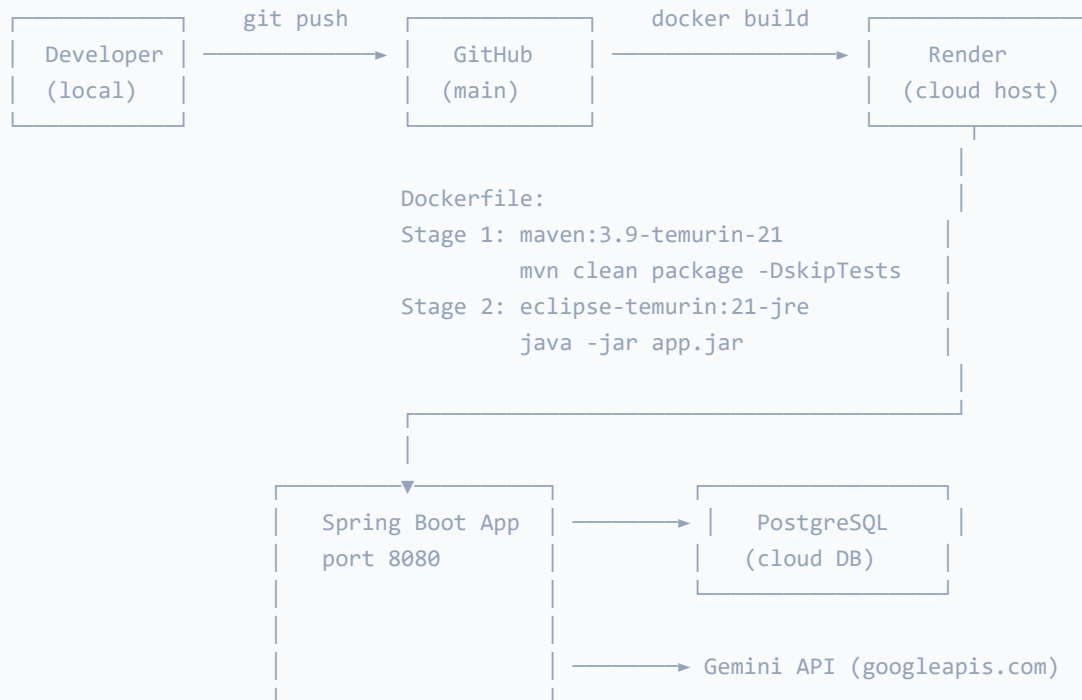
Module Pattern: Each JS file is a self-contained module using the IIFE or revealing module pattern. The `API` object exposes all REST methods. `auth.js` manages token lifecycle.

Page Initialization pattern (consistent across all pages):

```
document.addEventListener('DOMContentLoaded', () => {
  // 1. Auth Guard – check localStorage for token, redirect if missing
  // 2. Sidebar.init() – inject collapsible navigation
  // 3. VisualEngine.initAll() – theme, canvas, cursor glow, spotlights
  // 4. Page-specific data fetching via API.*() methods
  // 5. Event binding (forms, buttons, search)
});
```

API communication: `api.js` wraps `fetch()` calls. `auth.js` intercepts all `fetch` requests to inject the JWT Bearer header. On 401 responses (excluding auth/password endpoints), it auto-redirects to login.

2.11 Deployment Architecture



Environment variables: `DB_URL`, `DB_USERNAME`, `DB_PASSWORD`, `JWT_SECRET`, `GEMINI_API_KEY` — all injected at runtime, never committed to Git.

LEVEL 3

Interview Deep Dive

Defend your technical decisions. Answer difficult follow-up questions with confidence.

3.1 Java Questions

Q:

Testing: OOP principles, Dependency Inversion, Spring DI

A: Following the Dependency Inversion Principle — controllers depend on the `AuthService` interface, not `AuthServiceImpl`. This means I can swap implementations without touching controllers, and I can mock services in unit tests. Spring's IoC container injects the concrete implementation at runtime via `@RequiredArgsConstructor`.

⚡ **Follow-up:** What if you had multiple implementations of the same interface?

Q:

Design patterns, practical usage

A: Everywhere. Lombok's `@Builder` is on every entity (`User.builder().name("...").email("...").build()`), every DTO response (`ApiResponse.builder().status(200).message("...").data(...).build()`), and the JWT builder (`Jwts.builder().subject(email).signWith(key).compact()`). The Builder pattern is ideal when constructing objects with many optional fields — it's more readable than telescoping constructors.

⚡ **Follow-up:** How does Lombok's `@Builder` work under the hood?

Q:

Functional programming, practical Stream usage

A: Yes. For example in `ResumeServiceImpl.getUserResumes()`, I use `resumes.stream().map(resume -> ResumeHistoryResponse.builder()...build()).toList()` to transform entity lists into DTO lists. Also in

`InterviewQuestionServiceImpl` to map questions to response DTOs, and in `GlobalExceptionHandler` to collect field validation errors.

Q:

Spring IoC, constructor injection

A: I use constructor injection via Lombok's `@RequiredArgsConstructor`. All dependencies are declared as `final` fields, and Lombok generates a constructor that Spring uses to inject the beans. For example, `AuthServiceImpl` has `final UserRepository userRepository`, `final PasswordEncoder passwordEncoder`, `final JwtService jwtService` — all injected automatically.

⚡ **Follow-up:** Why constructor injection over field injection (`@Autowired`)?

3.2 Spring Boot Questions

Q:

Understanding of starter dependencies

A: `spring-boot-starter-web` (REST API, embedded Tomcat), `spring-boot-starter-data-jpa` (Hibernate + JPA), `spring-boot-starter-security` (Spring Security), `spring-boot-starter-validation` (Jakarta Bean Validation), `spring-boot-starter-actuator` (health/metrics), `spring-boot-starter-mail` (SMTP). Each starter brings a curated set of dependencies and auto-configuration.

Q:

Exception handling, AOP concepts

A: `@RestControllerAdvice` combines `@ControllerAdvice` and `@ResponseBody`. It acts as a global catch block for all controllers. When an exception is thrown from any controller, Spring checks the handler for a matching `@ExceptionHandler` method. My handler maps `InvalidCredentialsException` → 401, `ResourceNotFoundException` → 404, `GeminiApiException` → 503, `AccessDeniedException` → 403, `MethodArgumentNotValidException` → 400 with field-level error details. The catch-all `Exception.class` handler returns 500 and logs the full stack trace.

Q:

Configuration management, externalization

A: It's organized by concern: `spring.datasource` (PostgreSQL URL, credentials from env vars), `spring.jpa` (Hibernate dialect, `ddl-auto: update`), `server` (port 8080, context-path /api), `app.jwt`

(secret, expiration-ms: 86400000 = 24h), `app.gemini` (api-key, model, temperature: 0.7, max-output-tokens: 2048), `springdoc` (Swagger UI path). Sensitive values use `${ENV_VAR:default}` syntax for externalization.

3.3 Spring Security Questions

Q:

Deep security understanding

A: In `SecurityConfig.securityFilterChain()` : First, I enable CORS with a custom `CorsConfigurationSource` that allows specific origins. Then CSRF is disabled because it's a stateless REST API — no cookies or sessions to protect. Session management is set to `STATELESS` . A custom `JwtAuthenticationEntryPoint` handles 401 errors with JSON responses. Authorization rules: `/auth/**` , `/health/**` , `/swagger-ui/**` , and `/actuator/**` are `permitAll()` . `/admin/**` requires `hasRole("ADMIN")` . Everything else requires authentication. Finally, I add `JwtAuthenticationFilter` before `UsernamePasswordAuthenticationFilter` .

⚡ **Follow-up:** Why do you add JWT filter BEFORE `UsernamePasswordAuthenticationFilter`?

Q:

Fundamental security concepts

A: = "Who are you?" — verifying identity (JWT token validation, password check).
= "What can you do?" — checking permissions (`ROLE_USER` can access `/interviews`, only `ROLE_ADMIN` can access `/admin`). In VelloxPrep, authentication happens in `JwtAuthenticationFilter` ; authorization happens in `SecurityFilterChain` 's `.authorizeHttpRequests()` rules.

Q:

Understanding of CSRF attack vectors

A: CSRF (Cross-Site Request Forgery) exploits the browser's automatic cookie attachment. Since VelloxPrep uses JWT tokens in the `Authorization` header (not cookies), the browser doesn't automatically attach credentials to cross-site requests. Therefore CSRF protection is unnecessary and actually counterproductive for stateless REST APIs.

Q:

Error handling, security edge cases

A: In `JwtAuthenticationFilter`, `jwtService.isTokenValid(token)` calls `Jwts.parser().parseSignedClaims(token)`, which throws an `ExpiredJwtException`. This is caught by the generic `catch (Exception ex)` block, which logs a warning and lets the filter chain continue *without* setting authentication. Since no `SecurityContext` is populated, `SecurityFilterChain` rejects the request, and `JwtAuthenticationEntryPoint` returns a 401 JSON response.

3.4 JPA / Hibernate Questions

Q:

JPA relationships, cascade, fetch types

A: It's a `@OneToMany` from `User` to `InterviewSession` — one user has many sessions. In `User.java`: `@OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)`. The `mappedBy` says the `InterviewSession` entity owns the foreign key. `CascadeType.ALL` means save/delete operations propagate from `User` to their sessions. `orphanRemoval = true` means if a session is removed from the user's list, it gets deleted from the database. On the `InterviewSession` side: `@ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "user_id")`.

⚡ **Follow-up:** What is the N+1 problem and how does your project handle it?

Q:

Performance optimization, N+1 awareness

A: `@BatchSize(size = 20)` on `InterviewSession.interviewQuestions` tells Hibernate: when lazily loading questions for multiple sessions, fetch them in batches of 20 using `WHERE session_id IN (?, ?, ...)` instead of one query per session. This mitigates the N+1 problem — instead of 100 queries for 100 sessions, it takes ~5 queries.

3.5 PostgreSQL Questions

Q:

Database selection reasoning

A: The data model is inherently relational — users own sessions, sessions contain questions, questions have evaluations. These are well-defined relationships with referential integrity requirements. PostgreSQL was chosen over MySQL for its superior JSON support (useful if AI responses need flexible storage), better

standards compliance, and strong community. MongoDB was not appropriate because the data is structured with clear relationships — a document database would require embedding or manual joins.

3.6 REST API Questions

Q:

API design consistency

A: Every endpoint returns `ApiResponse<T>` — a generic wrapper with `status` (HTTP code), `message` (human-readable), `data` (payload, generic type T), and `timestamp`. Factory methods:

`ApiResponse.success("msg", data)`, `ApiResponse.created("msg")`, `ApiResponse.error(status, "msg")`. The `@JsonInclude(NON_NULL)` annotation ensures null fields are omitted from the JSON output. This gives clients a predictable structure for all responses — success and error.

Q:

HTTP semantics

A: `POST /auth/register` — creates a new resource (user). `POST /interviews/generate` — creates a new interview session. `PUT /users/password` — updates an existing resource (password change). `PUT /admin/users/{id}/role?role=ADMIN` — updates user's role. POST creates, PUT updates. DELETE removes (`DELETE /resumes/{id}`).

3.7 Gemini / AI Questions

Q:

Technology selection reasoning

A: Gemini was chosen for: (1) generous free tier for development and small-scale deployment, (2) native JSON-mode support (`responseMimeType: "application/json"`) which produces cleaner structured output, (3) Google's ecosystem integration, and (4) competitive quality for structured generation tasks like question creation.

Q:

Error handling, resilience

A: In `AnswerEvaluationServiceImpl`, the code first tries to locate JSON boundaries using `indexOf('{')` / `lastIndexOf('}')` to extract the JSON object even if surrounded by extra text. If no valid JSON object is found, it throws `GeminiApiException`. The `ObjectMapper.readTree()` call is wrapped in a try-catch — any parse failure throws `GeminiApiException("Failed to parse AI evaluation response")`. The `GlobalExceptionHandler` catches this and returns HTTP 503 to the client.

⚡ **Follow-up:** How would you improve this to be more resilient? (Retry logic, fallback responses, caching)

Q:

Security architecture

A: Three reasons: (1) — the key would be visible in browser DevTools/network tab. (2) — can't verify the user owns the session or has permission. (3) — generated questions need to be stored in the database, which requires server-side processing.

3.8 Resume / ATS Questions

Q:

Third-party library integration

A: Apache PDFBox is a Java library for working with PDF documents. In VelloxPrep, after a resume PDF is saved to disk, the `SkillExtractionService` and `AtsScoreService` use PDFBox to load the PDF and extract its text content. This extracted text is then sent to Gemini for AI analysis — skill extraction and ATS scoring. PDFBox handles the binary PDF format parsing, stripping formatting, and returning plain text.

3.9 Frontend Questions

Q:

Client-side auth implementation

A: `auth.js` stores the JWT in `localStorage` after login. It wraps the global `fetch` function to automatically inject `Authorization: Bearer {token}` on every request. On page load, each page checks if a token exists — if not, it redirects to `login.html`. The auth interceptor watches for 401 responses; if it receives one (excluding password-change and auth endpoints), it clears the token and redirects to login.

Q:*Performance optimization, architecture decisions*

A: `VisualEngine` is a centralized visual effects module in `components.js` that manages theme switching, particle canvas rendering, cursor glow, card spotlights, and scroll reveals. I centralized it because every page was duplicating the same canvas/cursor/theme code. The engine automatically detects touch devices (`'ontouchstart' in window || navigator.maxTouchPoints > 0`) and applies mobile optimizations — 14 particles instead of 80+, DPR capped at 1, disabled mouse tracking, and paused rendering during scrolling — to maintain 60fps on phones.

3.10 Docker / Deployment Questions

Q:*Containerization knowledge*

A: It's a multi-stage build. Uses `maven:3.9-eclipse-temurin-21-jammy` image, copies `pom.xml` and `src/`, runs `mvn clean package -DskipTests` to produce the JAR. Uses the lighter `eclipse-temurin:21-jre-jammy` image (no JDK, no Maven), copies only the built JAR, exposes port 8080, runs `java -jar app.jar`. The multi-stage approach reduces the final image size dramatically — the build tools stay in stage 1 and are discarded.

3.11 Project-Specific "Trap Questions" (30+)

Q:

A: VelloxPrep is a REST API consumed by a static frontend — there's no server-side session store. JWTs make the API stateless: every request carries its own authentication proof. This simplifies horizontal scaling (no shared session store needed) and makes the API consumable by any client (web, mobile, CLI).

Q:

A: `JwtAuthenticationFilter.doFilterInternal()` → calls `jwtService.isValid(token)` → which calls `Jwts.parser().verifyWith(signingKey).build().parseSignedClaims(token)`. This validates the signature (HMAC-SHA) and checks expiry.

Q:

A: An attacker could forge valid JWTs for any user/role. Mitigation: rotate the secret immediately (all existing tokens become invalid, forcing re-login). The secret is stored as an environment variable, never in source code.

Q:

A: SHA-256 is fast — that's bad for password hashing because GPUs can brute-force billions of SHA-256 hashes/second. BCrypt is intentionally slow (configurable rounds) and includes a random salt per hash, making rainbow table attacks impossible and brute-force impractical.

Q:

A: `SecurityFilterChain` checks `.hasRole("ADMIN")` → the user's authority is `ROLE_USER` → access denied → `AccessDeniedException` → `GlobalExceptionHandler` returns 403 Forbidden.

Q:

A: One-to-Many: `users.id` ← `interview_sessions.user_id` (foreign key). In JPA: `@ManyToOne(fetch = FetchType.LAZY)` on `InterviewSession`, `@OneToMany(mappedBy = "user")` on `User`. Cascade ALL with orphan removal — deleting a user deletes their sessions.

Q:

A: Gemini returns a JSON array of `{question, expectedAnswer}`. `InterviewQuestionServiceImpl` parses this with Jackson's `ObjectMapper.readTree()`, iterates the JSON array, creates `InterviewQuestion` entities, links them to the `InterviewSession` via `session.addQuestion()`, and saves everything with `sessionRepository.save(session)` — JPA cascades the save to all child questions.

Q:

A: `RestClientException` is thrown by Spring's `RestClient`. `GeminiServiceImpl` catches it and wraps it in `GeminiApiException`. `GlobalExceptionHandler` catches `GeminiApiException` and returns HTTP 503 (Service Unavailable) with an error message. The frontend shows the error to the user.
retry with exponential backoff, circuit breaker pattern.

Q:

A: Dependency Inversion Principle. Controllers depend on abstractions (interfaces), not implementations. Benefits: mockability in tests, swappable implementations (e.g., swap GeminiService for an OpenAI implementation without changing any controller), clean separation of contract vs implementation.

Q:

A: Set via `server.servlet.context-path: /api` in `application.yml`. This provides a clean namespace separation — all backend endpoints live under `/api/*`, making it easy to route at the reverse proxy level (e.g., `/api` → backend, everything else → frontend static files).

Q:

A: Gemini API rate limits and single-server architecture. (1) Horizontal scaling — since auth is stateless (JWT), add more Spring Boot instances behind a load balancer. (2) Database connection pooling (HikariCP is already included). (3) Redis cache for dashboard aggregations. (4) Queue AI requests through a message broker (RabbitMQ/Kafka) to handle rate limits. (5) CDN for frontend static assets.

Q:

A: The Gemini API call. Question generation takes 2-5 seconds depending on load. Each answer evaluation is another API call. For a 10-question interview, that's 11 sequential API calls. Improvements: batch evaluation requests, cache similar questions, pre-generate question pools.

Q:

A: `ResumeServiceImpl.validateFile()` checks: (1) file is not null/empty, (2) content type is exactly `application/pdf`, (3) file size ≤ 10MB. Files are saved with UUID-prefixed names to prevent path traversal. magic byte validation (check PDF header bytes), virus scanning, sandboxed parsing.

Q:

A: Simplicity for the current scale. The `uploads/` directory is a direct file system path. doesn't survive container restarts on Render's free tier. migrate to AWS S3 or Google Cloud Storage for persistent, scalable blob storage.

Q:

A: Potential race condition. Both transactions read the current score, calculate a new average, and write — the second write could overwrite the first. Current mitigation: `@Transactional` provides database-level

isolation. optimistic locking with `@Version` on the session entity, which throws `OptimisticLockingFailureException` on conflict.

Q:

A: VelloxPrep's frontend is page-based, not a single-page app. Each page has a specific purpose with relatively simple state. Vanilla JS gives full control over DOM, Canvas API, and performance without framework overhead (virtual DOM diffing, bundling complexity). For a larger team or more complex client-side routing, React would be justified.

Q:

A: `SecurityConfig.corsConfigurationSource()` allows specific origins (localhost:5500, render.com, vercel.app), specific methods (GET, POST, PUT, DELETE, etc.), specific headers (Authorization, Content-Type), and exposes the Authorization header. `allowCredentials(true)` allows cookies/auth headers in cross-origin requests.

Q:

A: The `loadUserByUsername(String email)` method queries `UserRepository.findByEmail()`. In VelloxPrep, email is the unique identifier for authentication — it's the JWT subject and the login credential. Spring Security's `UserDetails` interface requires a "username" concept; we map email to that role.

Q:

A: (1) Use Spring WebFlux for non-blocking Gemini API calls. (2) Add Redis caching for dashboard data. (3) Implement refresh token rotation. (4) Use AWS S3 for resume storage. (5) Add WebSocket for real-time evaluation feedback. (6) Implement a question cache to reduce Gemini API usage. (7) Add rate limiting to prevent API abuse.

Q:

A: Store refresh tokens in the database with a `revoked` flag. On token refresh: validate the refresh token, generate a new access + refresh token pair, revoke the old refresh token. If a revoked refresh token is used (indicating theft), revoke all tokens for that user.

Q:

A: Strictly speaking, `update` is convenient for development — Hibernate auto-creates/modifies tables. In production, `validate` or `none` is safer to prevent accidental schema changes. Database migrations should be handled by tools like Flyway or Liquibase. This is an area for improvement in VelloxPrep.

Q:

A: The key is injected via the `GEMINI_API_KEY` environment variable, referenced in `application.yml` as `${GEMINI_API_KEY:your_api_key_here}`. It's never committed to Git (listed in `.gitignore` via `.env`). On Render, it's set in the environment variables dashboard.

Q:

A: `@Transactional` wraps a method in a database transaction — if any exception occurs, all changes are rolled back. I use it on service methods like `register()`, `uploadResume()`, `submitAnswer()`. For read-only operations like `getUserResumes()`, I use `@Transactional(readOnly = true)` which hints Hibernate to optimize (skip dirty checking).

Q:

A: `ApiResponse<T>` uses Java generics — the `T` can be `LoginResponse`, `List<ResumeHistoryResponse>`, `Void`, or `Map<String, String>` (validation errors). Factory methods like `ApiResponse.success("msg", data)` use `<T>` type inference. `@JsonInclude(NON_NULL)` omits null data field on error responses.

Q:

A: `RestClient` is the modern replacement for `RestTemplate` introduced in Spring 6.1. It provides a fluent, builder-style API that's more readable. `RestTemplate` is in maintenance mode. `RestClient` also integrates better with the Spring ecosystem and supports the same underlying infrastructure.

Q:

A: It enables method-level security annotations like `@PreAuthorize("hasRole('ADMIN')")` on controller or service methods. This provides defense-in-depth — even if the URL-level configuration is misconfigured, method-level checks provide a second barrier.

Q:

A: Two mechanisms: (1) The system instruction explicitly says "respond ONLY with valid JSON" and provides the exact schema. (2) The `generationConfig` sets `responseMimeType: "application/json"`, which tells Gemini to constrain its output to valid JSON. The parsing code still includes defensive handling for edge cases.

Q:

A: On `@OneToMany(orphanRemoval = true)`: if an `InterviewQuestion` is removed from the session's question list (but the session itself isn't deleted), Hibernate will automatically DELETE that question from the database. Without it, the question would remain as an orphan row with no parent reference.

Q:

A: Currently, user input (skill name, answer text) is directly interpolated into prompts. A malicious user could inject "ignore previous instructions and..."

(1) Input sanitization — strip prompt-like keywords. (2) Use Gemini's system instruction separation (already implemented). (3) Validate AI output structure before trusting it. (4) Rate limit submissions.

Q:

A: `@ManyToOne(fetch = FetchType.LAZY)` on `InterviewSession`'s user field means the `User` entity is NOT loaded from the database when you load a session — only when you access `session.getUser()`. This prevents loading the full `User` object (with all their resumes and sessions) every time you query a single session. It's a critical performance optimization for entities with deep relationship graphs.

3.12 "Show Me the Code" Preparation

 MUST KNOW

FILE	WHY IT MATTERS	KEY THINGS TO EXPLAIN
<code>SecurityConfig.java</code>	The heart of your security setup	CORS, CSRF disabled, stateless sessions, filter chain order, <code>hasRole("ADMIN")</code> , JWT filter position
<code>JwtService.java</code>	Core JWT operations	<code>generateToken()</code> , <code>extractEmail()</code> , <code>isTokenValid()</code> , HMAC-SHA signing, expiration handling

<code>JwtAuthenticationFilter.java</code>	Request-level security	OncePerRequestFilter, 7-step authentication flow, SecurityContext population
<code>AuthServiceImpl.java</code>	Login/register flow	BCrypt hashing, password matching, JWT generation on successful login
<code>GeminiServiceImpl.java</code>	AI integration centerpiece	RestClient usage, request body construction, response parsing, error handling
<code>GlobalExceptionHandler.java</code>	Error handling strategy	@RestControllerAdvice, specific exception → HTTP status mapping
<code>User.java</code>	Core entity with all relationships	@Entity, @OneToMany, CascadeType.ALL, @Builder, @CreationTimestamp
<code>ApiResponse.java</code>	Response envelope pattern	Generics, factory methods, @JsonInclude

● SHOULD KNOW

FILE	WHY IT MATTERS
<code>InterviewQuestionServiceImpl.java</code>	End-to-end AI feature: prompt → Gemini → parse → persist
<code>AnswerEvaluationServiceImpl.java</code>	AI evaluation with JSON extraction, score calculation, progress tracking
<code>ResumeServiceImpl.java</code>	File upload, validation, PDFBox integration, ATS scoring
<code>CustomUserDetailsService.java</code>	Spring Security's bridge to your User entity
<code>application.yml</code>	All configuration in one place
<code>frontend/js/auth.js</code>	Client-side JWT management
<code>frontend/js/api.js</code>	REST client architecture

● NICE TO KNOW

FILE	WHY IT MATTERS
------	----------------

<code>AdminController.java</code>	Role-based endpoint access
<code>Dockerfile</code>	Multi-stage build understanding
<code>components.js (VisualEngine)</code>	Mobile performance optimization
<code>pom.xml</code>	Dependency management

3.13 AI-Assisted Development / Vibe Coding Questions

HONESTY IS STRENGTH

AI-assisted development is a professional skill. The question isn't whether you used AI — it's whether you [understand](#) what was built and can [own](#) it technically.

Q:

A: "Yes, I used AI as a development accelerator. I made the architectural decisions — choosing Spring Boot with JWT authentication, the service-interface pattern, PostgreSQL, Gemini AI integration. AI helped me with implementation speed, especially for boilerplate code, CSS styling, and initial scaffolding. But I debugged every issue myself, understood every dependency, and made every design trade-off. I can explain any class in this codebase because I reviewed and integrated every piece."

Q:

A: "The system design and architecture — deciding on JPA entities and their relationships, the security model, the AI integration strategy with structured prompts, the DTO separation pattern, the mobile performance optimization approach. I also did all debugging, deployment configuration, and iterative design decisions based on testing."

Q:

A: [This is where Level 2 and Level 3 preparation pays off. You should be able to walk through any MUST KNOW file line by line.]

Q:

A: "Several issues required manual debugging. For example, the global auth interceptor was treating password-change 401 responses as expired tokens and redirecting to login — I had to add endpoint exclusion logic. The Gemini response parser initially broke on responses with markdown code fences — I added defensive JSON extraction with indexOf/lastIndexOf. These are the kind of integration issues that only surface during end-to-end testing."

Q:

A: "Yes, but it would take significantly longer. AI accelerated development maybe 3-5x. Without it, I'd still make the same architectural decisions, but writing boilerplate JPA entities, CSS animations, and the Gemini REST client would take much more time. The core skill isn't typing code — it's understanding the system, making design decisions, and debugging integration points."

3.14 Final Interview Checklist



MUST MASTER

- JWT authentication flow (registration → login → token → filter → context)
- SecurityFilterChain configuration (CORS, CSRF, stateless, endpoint rules)
- JwtAuthenticationFilter's 7-step process
- GeminiServiceImpl: request building, API call, response parsing, error handling
- Entity relationships (User → Session → Question → Evaluation)
- Service-interface pattern and why it exists
- GlobalExceptionHandler and HTTP status mapping
- ApiResponse generic wrapper pattern
- How frontend communicates with backend (auth.js + api.js)
- Multi-stage Dockerfile
- 60-second and 2-minute project explanations



MUST UNDERSTAND

- BCrypt vs SHA-256 for password hashing
- Why CSRF is disabled in stateless REST APIs
- N+1 problem and @BatchSize optimization
- CascadeType.ALL and orphanRemoval behavior
- @Transactional and readOnly optimization
- Prompt engineering for structured AI output
- PDF parsing with PDFBox
- Mobile performance optimization strategy
- CORS and why it's needed
- Environment variable externalization

● VERIFY BEFORE INTERVIEW

- OAuth implementation (OAuthController.java) — verify it's fully functional end-to-end
- Refresh token implementation — the config mentions refresh-expiration-ms but verify if rotation is implemented
- MapStruct usage — verify it's actually used in DTO mapping (may just be a dependency)
- AtsScoreService and SkillExtractionService — verify their actual Gemini prompts and logic
- CodingTestService — verify the code evaluation flow (run/submit/finish lifecycle)
- McqService — verify MCQ generation and answer validation logic
- Mail functionality — starter is included but verify if any email sending is implemented
- Spring Actuator metrics — verify what's actually exposed and useful

3.15 Final Mock Interview (25 Questions)

Try answering each question WITHOUT looking at the answer key below. Time yourself.

Q1. Tell me about your project in 60 seconds.

Q2. Who is the target user and what problem does it solve?

Q3. What technologies did you use and why?

Q4. What makes VelloxPrep different from just reading interview questions online?

Q5. Walk me through the key features.

Q6. Draw the high-level architecture on a whiteboard.

Q7. Trace a request from the browser to the database and back.

Q8. How does your backend layer architecture work? Why separate controllers and services?

Q9. Explain your database schema. What are the key relationships?

Q10. How does your AI integration work? Trace the Gemini API call.

Q11. Explain dependency injection in Spring Boot. How do you use it?

Q12. What is `@RestControllerAdvice` and how does your exception handling work?

Q13. Explain the Builder pattern. Where do you use it?

Q14. What Spring Boot starters do you use and what does each provide?

Q15. How does your `application.yml` externalize sensitive configuration?

Q16. Walk me through the complete JWT authentication flow — from registration to authenticated request.

Q17. What is the SecurityFilterChain and how is yours configured?

Q18. Explain JPA entity relationships with an example from your project.

Q19. What happens if Gemini returns invalid or empty JSON?

Q20. How would you scale this application to handle 100,000 users?

Q21. Why is CSRF disabled? Isn't that a security risk?

Q22. What's the biggest limitation of your current architecture?

Q23. How do you handle concurrent requests updating the same session?

Q24. Did you use AI to build this? Explain your development process honestly.

Q25. If you rebuilt this project from scratch, what would you do differently?

Mock Interview Answer Key

Q1. See Section 1.5 (60-Second Answer).

Q2. Target users are software engineering candidates. Problem: interview prep is passive — no personalization, no feedback, no progress tracking. VelloxPrep provides AI-generated questions, real-time evaluation, and performance analytics.

- Q3 See Section 1.4 (Technology Stack table). Key choices: Java 21 for enterprise robustness, Spring Boot for rapid development, PostgreSQL for relational data integrity, Gemini AI for question generation and evaluation.
- Q4 Three differentiators: (1) Questions generated from YOUR resume. (2) AI evaluates free-text answers with scores + feedback. (3) Dashboard tracks improvement over time.
- Q5 See Section 1.3 (Major Features).
- Q6 See Section 2.1 (Architecture Diagram). Frontend → REST API → Security → Controllers → Services → Repositories → PostgreSQL. External: Services → Gemini API, ResumeService → PDFBox.
- Q7 See Section 2.2 (Request Lifecycle). JWT filter → security chain → controller → service → Gemini + repository → ApiResponse → client.
- Q8 See Section 2.3. Controllers handle HTTP mapping only. Services contain business logic. This follows Single Responsibility and enables unit testing.
- Q9 See Section 2.5. User 1:N InterviewSession 1:N InterviewQuestion 1:1 AnswerEvaluation. User 1:N Resume. User 1:N McqSession/CodingSession/Progress.
- Q10 See Section 2.7. Service builds JSON prompt → RestClient POSTs to Gemini API → parses candidates[0].content.parts[0].text → Jackson deserializes → entities persisted.
- Q11 Constructor injection via @RequiredArgsConstructor. All dependencies declared as final fields. Spring IoC container instantiates and injects beans automatically.
- Q12 See Section 3.2 (Spring Boot Q2). @RestControllerAdvice globally catches exceptions. Each @ExceptionHandler maps to a specific HTTP status and ApiResponse.
- Q13 See Section 3.1 (Java Q2). Lombok @Builder on entities and DTOs. Also Jwts.builder() for JWT creation. Ideal for objects with many optional fields.
- Q14 See Section 3.2 (Spring Boot Q1). web, data-jpa, security, validation, actuator, mail.
- Q15 \${ENV_VAR:default} syntax. Sensitive values (DB password, JWT secret, Gemini key) are environment variables, never hardcoded.
- Q16 See Section 2.6 (complete 7-step flow).
- Q17 See Section 3.3 (Security Q1). CORS → CSRF disabled → Stateless → JWT entry point → permit /auth → require ADMIN for /admin → authenticate all else → JWT filter before UsernamePasswordAuthenticationFilter.

Q18 See Section 3.4 (JPA Q1). User @OneToMany(cascade=ALL, orphanRemoval=true) → InterviewSession @ManyToOne(fetch=LAZY).

Q19 See Section 3.7 (AI Q2). indexOf/lastIndexOf extraction → ObjectMapper parsing → GeminiApiException → GlobalExceptionHandler → 503.

Q20 See Trap Q11. Horizontal scaling (stateless JWT), connection pooling, Redis cache, message queue for AI requests, CDN.

Q21 See Section 3.3 (Security Q3). JWT in Authorization header, not cookies. Browser doesn't auto-attach headers to cross-origin requests.

Q22 Gemini API latency (2-5s per call, 11 calls per full interview). File storage on local disk (not persistent across deployments). No caching layer.

Q23 See Trap Q15. @Transactional provides isolation. Improvement: @Version for optimistic locking.

Q24 See Section 3.13. Honest: AI accelerated development. You made architectural decisions, debugged integration issues, own the codebase.

Q25 See Trap Q19. WebFlux, Redis, S3, refresh token rotation, WebSocket, question cache, rate limiting.